COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	7/16/26	**DATE SUBMITTED:**	7/20 /2026
NAME:	Samuel D. Ramos	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. _2.1_ [Hands-on Activity 2.1 Arrays, Pointers and Dynamic Memory Allocation]

1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        // constructor
        Student(std::string newName="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor called." << std::endl;
        }

        // Deconstructor
        ~Student(){
            std::cout << "Deconstrutor Called" << std::endl;
        }

        // Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        // Display Attributes
        void printDetails(){
            std::cout << this ->studentName << " " << this -> studentAge << std::endl;
        }

};

int main(){
```

```
    Student student1 ("Roman",28);
    Student student2 (student1);
    Student student3;
    student3 = student2;

    return 0;


}
```

OUTPUT

```
Constructor called.
Copy Constructor Called
Constructor called.
Deconstrutor Called
Deconstrutor Called
Deconstrutor Called
PS C:\Users\TIPQC\a>
```

Observation	- In the main function, it first uses the constructor to store the details about student 1. Student 2 is then created from student 1. Student 3 is then created. This leads to the notification of the constructor and copy constructor being called. After the code reaches return 0;, the destructor automatically functions to free up space.

Table 2-1. Initial Driver Program

Screenshot

```
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        // constructor
        Student(std::string newName="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor called." << std::endl;
        }

        // Deconstructor
        ~Student(){
            std::cout << "Deconstrutor Called" << std::endl;
        }

        // Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        // Display Attributes
        void printDetails(){
            std::cout << this ->studentName << " " << this -> studentAge << std::endl;
        }

};

int main(){
    const size_t j = 5;

    Student studentList[j]= {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

OUTPUT

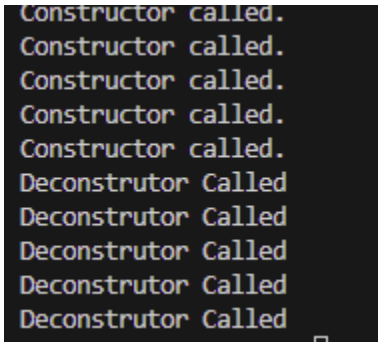
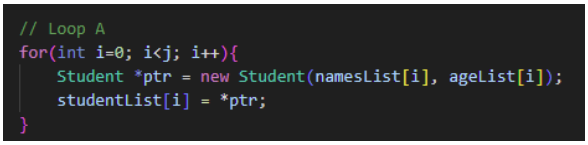
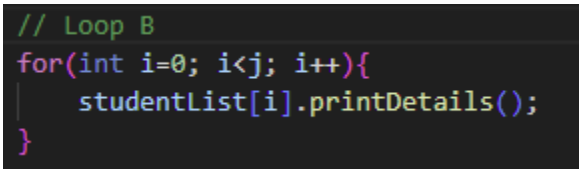
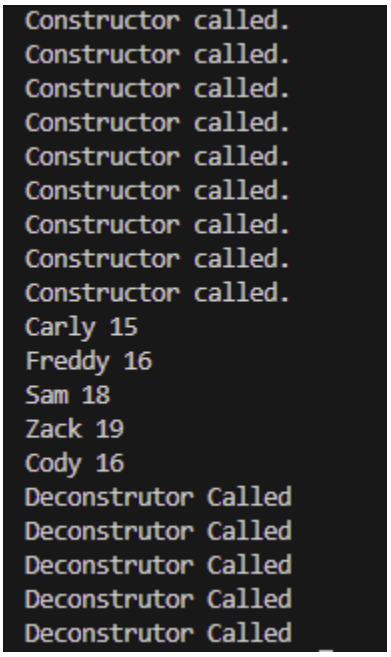
	
Observation	- The Constructor is called 5 times, it fills the array studentList. The array namesList and ageList are then initialized. At the end of the code, the constructor functions 5 times to free up space.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	- This loop stores the memory addresses of each student in the arrays namesList and ageList. After that, the values taken from the previous arrays are now stored in the array studentList.
Loop B	
Observation	- This loop happens after Loop A has completed its loop. It uses the array studentList and prints out the elements it contains using the function printDetails().

Output	
Observation	- The constructor is called to fill the array studentList five times, it is replaced by the values of namesList and ageList array. The values of the studentList array is then printed out, and after reaching return 0; in the code, the deconstructor automatically functions to free up space.
Modifications	

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        // constructor
        Student(std::string newName="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor called." << std::endl;
        }

        // Deconstructor
        ~Student(){
            std::cout << "Deconstrutor Called" << std::endl;
        }

        // Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        // Display Attributes
        void printDetails(){
            std::cout << this ->studentName << " " << this -> studentAge << std::endl;
```

	
Observations	- The constructor function is used to construct the received data which is then stored into an array (studentList), and the initialized arrays' values (namesList and ageList) are the stored in the studentList using pointers in Loop A. in Loop B, the values stored in studentList are printed out. After reaching return 0;, The deconstructor automatically functions to free up space.

Table 2-3 and 2-4.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM #1

```cpp
// Problem 1
class GroceryList{
    private:
        std::string name;
        int price;
        int quantity;

    public:
        // Constructor
        GroceryList(std::string productName, int productPrice, int productQuantity){
            name = productName;
            price = productPrice;
            quantity = productQuantity;
        };

        // Deconstructor
        virtual ~GroceryList(){
            std::cout << "Destructor Was Called" << std::endl;
        };

        // Copy Constructor
        GroceryList(const GroceryList &other){
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        };
```

```
29
30          // Copy Assignment Operator
31          GroceryList& operator = (const GroceryList& other){
32              if (this!= &other){
33                  name = other.name;
34                  price = other.price;
35                  quantity = other.quantity;
36              }
37              return *this;
38          }
39
40          int calculateSum(){
41              return price*quantity;
42          }
43
44          void display(){
45              std::cout << name << "\tPHP " << price << "\tx" << quantity << std::endl;
46          }
47      };
48
49      class Fruit:public GroceryList{
50          public:
51          // Constructor
52          Fruit(std::string productName, int productPrice, int productQuantity)
53              : GroceryList(productName, productPrice, productQuantity){}
54
```

```
    // Destructor
    ~Fruit() override{
        std::cout << "Fruit Destroyed, ";
    }

    // Copy Constructor
    Fruit(const Fruit& other)
        : GroceryList(other){}

    // Copy Assignment Operator
    Fruit& operator = (const Fruit& other){
        GroceryList::operator = (other);
        return *this;
    }
};

class Vegetable:public GroceryList{
    public:
    // Constructor
    Vegetable(std::string productName, int productPrice, int productQuantity)
        : GroceryList(productName, productPrice, productQuantity){}

    // Destructor
    ~Vegetable() override{
        std::cout << "Vegetable Destroyed, ";
    }
```

```

    // Copy Constructor
    Vegetable(const Vegetable& other)
        : GroceryList(other){}

    // Copy Assignment Operator
    Vegetable& operator = (const Vegetable& other){
        GroceryList::operator = (other);
        return *this;
    }
};
```

- Constructor, Destructor, Copy Constructor, and Copy Assignment Operator were made for each classes. The override and virtual were used so that the notification for the classes Vegetable and Fruits would go first, before the destructor of class GroceryList

PROBLEM #2

```
int main(){
    // Problem 2
    GroceryList* groceryList[4];

    groceryList[0] = new Fruit("Apple", 10 ,7);
    groceryList[1] = new Fruit("Banana", 10, 8);
    groceryList[2] = new Vegetable("Broccoli", 60, 12);
    groceryList[3] = new Vegetable("Lettce", 50, 10);

    // Output Complete List
    std::cout << "Grocery List:" << std::endl;
    for (int i=0; i<4; i++){
        groceryList[i] -> display();
    }

    std::cout << "Total: PHP " << totalSum(groceryList, 4) << std::endl << std::endl;
```

- Creates an array which contains the products from the grocery list, indicating what class the products belong to. The grocery list is displayed along with the total.

PROBLEM #3

```
94    //Problem 3
95    int totalSum(GroceryList* list[], int size);
96
```

```
140   // Problem 3
141   int totalSum(GroceryList* list[], int size){
142       int total = 0;
143
144       for(int i=0; i<size; i++){
145           total += list[i] -> calculateSum();
146       }
147       return total;
148   }
```

- Initializes the function above the main function, then defines it outside the main function.

PROBLEM #4

```
116     // Problem 4
117     delete groceryList[3];
118     groceryList[3] = nullptr;
119     std::cout << "\n";
120 
121     // Output with lettuce deleted
122     std::cout << "Grocery List:" << std::endl;
123     for (int i=0; i<3; i++){
124         groceryList[i] -> display();
125     }
126 
127     std::cout << "Total: PHP " << totalSum(groceryList, 3) << std::endl << std::endl;
128 
129     std::cout << "\n";
130 
131     for (int i=0; i<4; i++){
132         delete groceryList[i];
133         std::cout << "\n";
134     }
135 
```

- **First deletes the lettuce and sets it to nullptr. The updated list is then displayed along with its total, then the destructor functions to free up space.**

OUTPUT

```
Grocery List:
Apple   PHP 10  x7
Banana  PHP 10  x8
Broccoli        PHP 60  x12
Lettce  PHP 50  x10
Total: PHP 1370

Vegetable Destroyed, Destructor Was Called

Grocery List:
Apple   PHP 10  x7
Banana  PHP 10  x8
Broccoli        PHP 60  x12
Total: PHP 870


Fruit Destroyed, Destructor Was Called

Fruit Destroyed, Destructor Was Called

Vegetable Destroyed, Destructor Was Called
```

CONCLUSION:

- This hands-on activity trained my skills on coding with c++ using constructor, destructor, copy constructor, copy assignment operator, and using them in different classes. I had trouble forming the array which contains the list of products, which will serve as a lesson on which skills I need to improve on, and what I need to master. I also still haven't fully memorized the formats of the copy constructor and the copy assignment operator which led to me relying on my notes, giving me yet another idea on what I need to work on.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, formatting

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Assistance with the notification for destructor, setting the array for grocerylist, displaying the grocerylist, and formatting of copy constructor and copy assignment operator.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.